

计算机图形学

实验七： 国产开发平台的使用

实验内容

- 使用MindSpore框架进行深度学习网络DCGAN搭建，利用动漫头像数据集进行模型训练、图像生成；
- 通过本实验了解深度学习任务开发的具体流程，包括：数据处理、创建网络、连接网络和损失函数 损失函数和优化器和模型训练及图像生成；
- 了解经典神经网络DCGAN的结构特点和创新设计，以及深度学习网络搭建的重要概念。

开发平台介绍

- 昇腾910 (Ascend 910) 是华为自研的一款基于达芬奇架构的AI处理器，具有超高算力的和极高的能效比，最高可达320TFLOPS (FP16) 的浮点计算能力。昇腾910的片上系统 (SoC) 还集成了多个CPU、DVPP和任务调度器，因而具有自我管理能力，可以充分发挥其高算力的优势。强大的矩阵、向量运行能力，在进行海量数据运算的神经网络训练场景有巨大的优势。
- 昇思MindSpore (官方网站: <https://www.mindspore.cn/>) 是一种适用于端边云场景的新型开源深度学习训练/推理框架。MindSpore提供了友好的设计和高效的执行，旨在提升数据科学家和算法工程师的开发体验，并为Ascend AI处理器提供原生支持，以及软硬件协同优化。同时，MindSpore作为全球AI开源社区，致力于进一步开发和丰富AI软硬件应用生态。
- ModelArts (官方网站: <https://console.huaweicloud.com/modelarts/>) 是面向AI开发者的一站式开发平台，提供海量数据预处理及半自动化标注、大规模分布式训练、自动化模型生成及模型按需部署能力，帮助用户快速创建和部署AI应用，管理全周期AI工作流。

算法介绍-GAN

- GAN由Ian Goodfellow于2014年发明，并在论文Generative Adversarial Nets中首次进行了描述，GAN由两个不同的模型组成：生成器和判别器：
- 生成器的任务是生成看起来像训练图像的“假”图像；
- 判别器需要判断从生成器输出的图像是真实的训练图像还是虚假的图像。
- 在训练过程中，生成器会不断尝试通过生成更好的假图像来骗过判别器，而判别器在这过程中也会逐步提升判别能力。这种博弈的平衡点是，当生成器生成的假图像和训练数据图像的分布完全一致时，判别器拥有50%的真假判断置信度。

算法介绍-DCGAN

- DCGAN（深度卷积对抗生成网络）是GAN的直接扩展。不同之处在于，DCGAN会分别在判别器和生成器中使用卷积和卷积转置层。
- 它最早由Radford等人在论文Unsupervised Representation Learning With Deep Convolutional Generative Adversarial Networks中进行描述。判别器由分层的卷积层、BatchNorm层和LeakyReLU激活层组成。输入是3x64x64的图像，输出是该图像为真图像的概率。生成器则是由转置卷积层、BatchNorm层和ReLU激活层组成。输入是标准正态分布中提取出的隐向量，输出是3x64x64的RGB图像。
- 本节课将使用70,171张动漫头像数据集来训练一个生成对抗网络，接着使用该网络生成动漫头像图片。

实验环境配置

- 进入华为云ModelArts控制台

<https://www.huaweicloud.com/product/modelarts.html>

- 在华为云ModelArts主页，点击“管理控制台”进入ModelArts的管理页面。



实验环境配置

- 点击此处进入授权界面。



实验环境配置

- 点击新增委托。

华为云

控制台

华北-北京四

搜索云服务、资源(IP / 名称 / ID)、快捷操作... 备案 资源 费用 企业

AI开发平台ModelArts / 权限管理 / 添加授权

< 添加授权

授权配置

授权对象类型 ?

IAM子用户

联邦用户

委托用户

所有用户

授权对象

hw073303232

委托选择

已有委托

新增委托

查看和配置更多权限，请访问IAM服务进行配置。[立即前往](#)

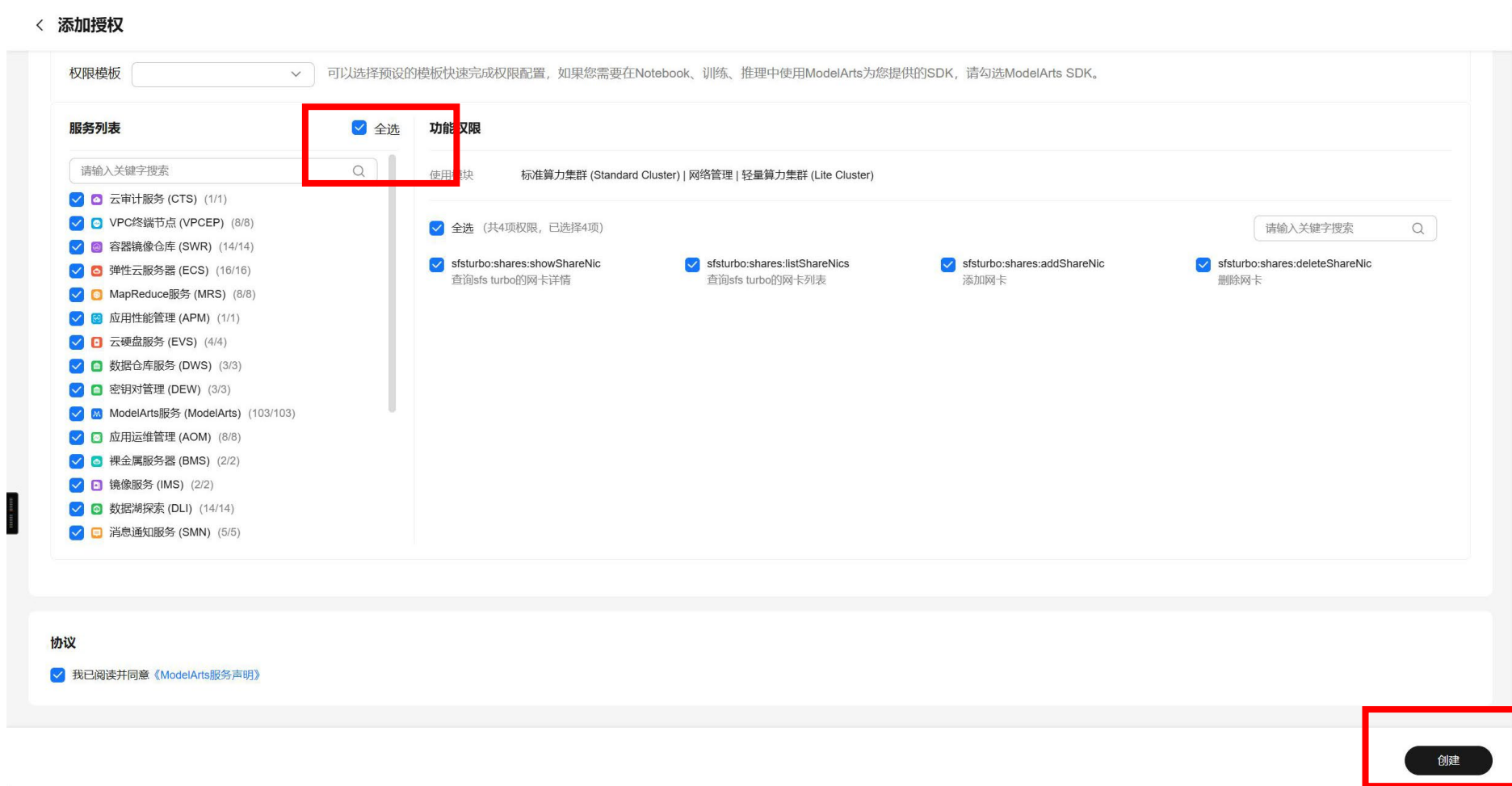
委托名称 ?

modelarts_agency

您最多可以创建50个委托，您还可以创建49个委托。

实验环境配置

- 服务列表点击全选，然后点击创建。



实验环境配置

- 控制台区域选择“华北-北京四”或者“乌兰察布一”，在左侧菜单栏中选择“开发空间”的“Notebook”。



实验环境配置

- 在网页右上角，点击“创建”按钮，创建一个新的Notebook，其配置如下：
- 名称：自定义。
- 自动停止：打开，默认1小时。
- 镜像：选择“公共镜像”，在搜索栏搜索含“mindspore”的镜像，选择MindSpore2.1版本、Ascend+ARM的基础镜像。
- 类型：ASCEND。
- 规格：Ascend: 1*Ascend910|CPU: 24核 96GB。
- 存储配置：云硬盘EVS，默认5GB，可根据实验需要自定义。注意：云硬盘从Notebook实例创建成功起，直至删除成功，每GB按照规定费用收费。

实验环境配置

★ 自动停止

开启该选项后, 该Notebook实例将在运行时长超出您所选择的时长后, 自动停止。

1 小时

2 小时

4 小时

6 小时

自定义

★ 镜像

公共镜像

自定义镜像

mindspore

×

Q

C

名称	描述
mindspore_2.1.0-cann_6.3.2-py_3.7-euler_2.10.7-aarch64-d910	Ascend+ARM algorithm development and training. MindSpore is preset i...
mindspore1.7.0-cuda10.1-py3.7-ubuntu18.04	GPU general algorithm development and training, preconfigured with AI...
mindspore1.7.0-py3.7-ubuntu18.04	CPU general algorithm development and training, preconfigured with AI...
tensorflow1.15-mindspore1.5.1-cann5.0.3-euler2.8-aarch64	Ascend+ARM算法开发和训练基础镜像, AI引擎预置TensorFlow和MindSp...
mindspore1.2.0-cuda10.1-cudnn7-ubuntu18.04	GPU算法开发和训练基础镜像, 预置AI引擎MindSpore-GPU
mindspore1.2.0-openmpi2.1.1-ubuntu18.04	CPU算法开发和训练基础镜像, 预置AI引擎MindSpore-CPU

10

总条数: 6

<

1

>

实验环境配置

* 资源类型

公共资源池 专属资源池 DevServer

* 类型

ASCEND

* 规格

Ascend: 1*Ascend910|ARM: 24核 96GB
昇腾910(32GB显存)单卡规格, 配搭ARM处理器, 适合深度学习场景下的模型训练和调测

* 存储配置

云硬盘EVS ?
云硬盘EVS作为持久化存储挂载在/home/ma-user/work目录下, 该目录下的内容在实例停止后会被保留, 存储支持在线按需扩容

* 磁盘规格

- 5 + GB 自定义

! 磁盘规格默认为5GB, 从Notebook实例创建成功起, 直至删除成功, 每GB按照规定费用收费。

SSH远程开发 ☐

配置费用: **¥19.507**/小时
🎁 优先扣减免费套餐用量, [了解更多](#)

实验环境配置

- 配置完成之后“立即创建”，规格确认无误之后“提交”。

产品名称	产品规格	计费模式	价格
notebook-9d64	描述	--	
	自动停止	1 小时	
	镜像	mindspore_2.1.0-cann_6.3.2-py_3.7-euler_2.10.7-aarch64-d910	
	资源类型	公共资源池	
	规格	Ascend: 1*Ascend910 ARM: 24核 96GB	
	存储配置	云硬盘EVS	
	存储空间	5 GB	
	SSH远程开发	--	
	远程访问白名单	--	
	标签	--	
			Notebook:¥19.50/小时 云硬盘EVS:¥0.007/小时
费用: ¥19.507 /小时 优先扣减免费套餐用量, 了解更多			上一步提交

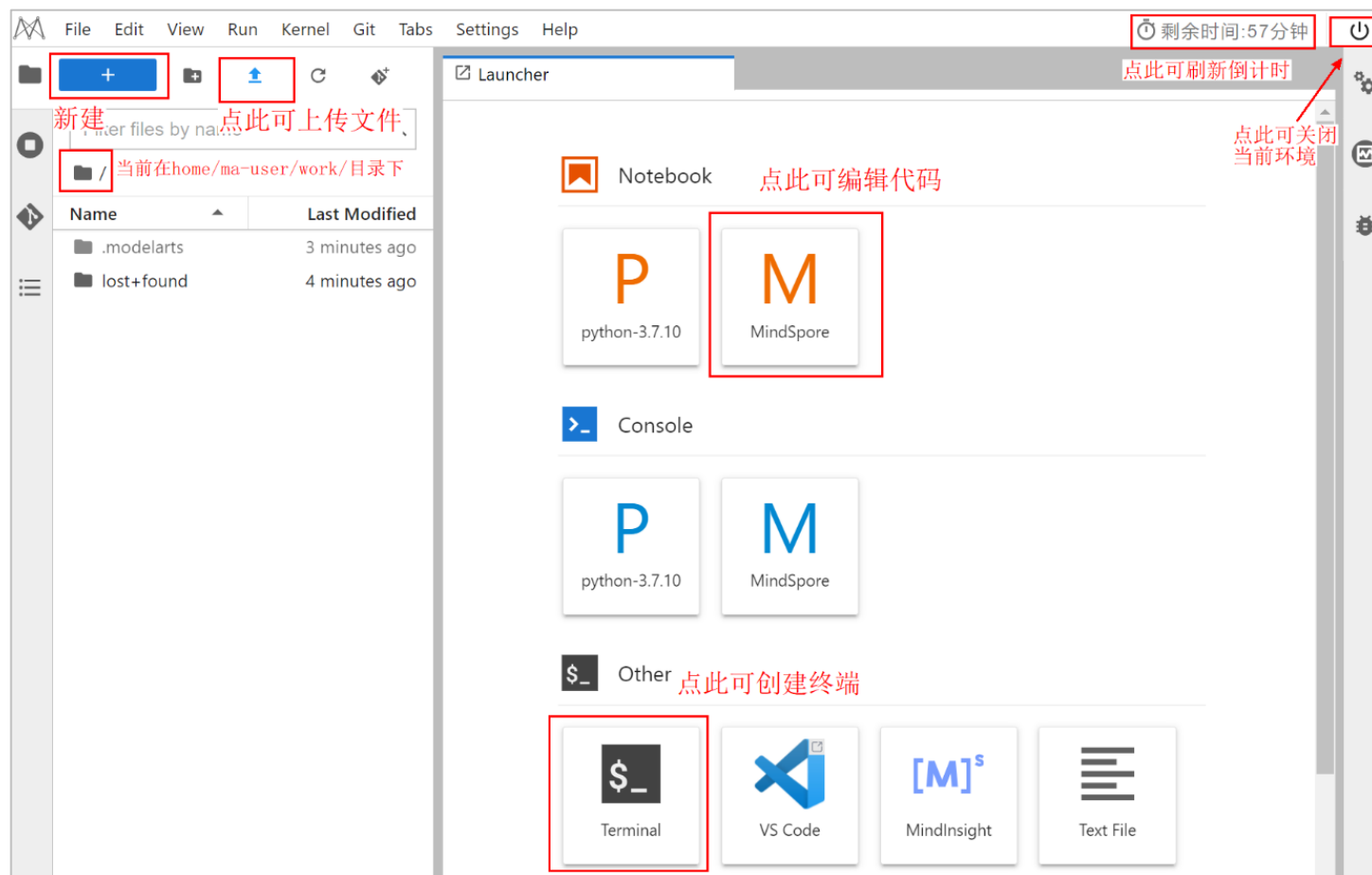
实验环境配置

- 当上一步创建好Notebook状态显示为“运行中”时，在右侧操作中“打开”，即可进入在线编程页面。

名称 ▾	状态 ▾	镜像 ▾	规格 ▾	描述 ▾	操作
notebook-9d64 fafa30cb-cc1a-4777-9657-aea21a9...	→ 运行中 (58分钟)	mindspore_2.1.0-cann_6.3.2-p...	Ascend: 1*Ascend910 ARM: 24核 96GB	--	打开 启动 停止 更多 ▾

实验环境配置

- 可以在此页面创建或编辑MindSpore的项目， 如图所示：



实验环境配置

- 实验完成之后，请及时关闭或删除Notebook训练作业，避免产生不必要的资源浪费。

名称 ▾	状态 ▾	镜像 ▾	规格 ▾	操作
notebook-9d64 fafa30cb-cc1a-4777-9657-aea21a9...	→ 运行中 (48分)	mindspore_2.1.0-cann_6.3.2-py_3.7-euler...	Ascend: 1*Ascend910 ARM: 24核 96GB	打开 启动 停止 更多 ▾

名称 ▾	状态 ▾	镜像 ▾	规格 ▾	操作
notebook-9d64 fafa30cb-cc1a-4777-9657-aea21a9...	→ 运行中 (43分)	mindspore_2.1.0-cann_6.3.2-py_3.7-euler...	Ascend: 1*Ascend910 ARM: 24核 96GB	打开 启动 停止 更多 ▲
notebook-7f09 fafa30cb-63ac-4e3d-bd4e-064264d...	ⓘ 停止	mindspore_2.1.0-cann_6.3.2-py_3.7-euler...	Ascend: 1*Ascend910 ARM: 24核 96GB	打开 启动 删除 变更镜像 变更规格 保存镜像 VS Code接入
llama fafa30cb-462d-49f3-9169-48ede30...	ⓘ 停止	mindspore_2.1.0-cann_6.3.2-py_3.7-euler...	Ascend: 1*Ascend910 ARM: 24核 96GB	打开 启动

实验步骤-数据下载与处理

- 步骤一： 下载实验所需模块（代码参照实验手册第5页2.2章步骤1）
- 开始实验之前， 需要首先更新环境中的依赖包， 在Notebook中输入以下命令并运行。

```
# 安装过程可能会有异常提示， 不影响实验操作
```

```
!pip install download
```

```
%env no_proxy='a.test.com,127.0.0.1,2.2.2.2'
```

实验步骤-数据下载与处理

- 步骤二：数据下载（代码参照实验手册第6页2.2章步骤2）
- 我们使用的动漫头像数据集共有70,171张动漫头像图片，图片大小均为96*96。首先我们将数据集下载到指定目录下并解压。
- 示例代码如下：

```
from download import download  
  
url = "https://download.mindspore.cn/dataset/Faces/faces.zip"  
path = download(url, "./faces", kind="zip", replace=True)
```

实验步骤-数据下载与处理

- 步骤二： 数据下载
- 下载后的数据集目录结构如下：

faces/

├── faces

│ ├── 0.jpg

│ ├── 1.jpg

│ ├── 2.jpg

│ └── ...

│ └── 70169.jpg

└── 70170.jpg

实验步骤-数据下载与处理

- 步骤三： 数据处理相关的一些输入定义：

参数	参数说明
batch_size	训练中使用的批量大小， DCGAN论文使用的批量大小为128；
image_size	训练图像的大小， 此实现默认为`64x64`， 如果需要其他尺寸， 则必须同时更改`D`和`G`的结构；
nc	输入图像中的彩色通道数， 因为此次是彩色图像所以设为3；
nz	隐向量的长度；
ngf	设置通过生成器的特征图的深度；
ndf	设置通过判别器传播的特征图的深度；
num_epochs	要运行的训练周期数， 训练更长的时间可能会导致更好的结果， 但也会花费更长的时间；
lr	训练的学习率；
beta1	Adam优化器的`beta1`超参数。如DCGAN论文所述， 该数字应为0.5；

实验步骤-数据下载与处理

- 步骤三： 数据处理相关的一些输入定义： （代码参照实验手册第6页2.2章步骤3）

```
batch_size = 128      # 批量大小
image_size = 64       # 训练图像空间大小
nc = 3                # 图像彩色通道数
nz = 100              # 隐向量的长度
ngf = 64              # 特征图在生成器中的大小
ndf = 64              # 特征图在判别器中的大小
num_epochs = 10       # 训练周期数
lr = 0.0002           # 学习率
beta1 = 0.5           # Adam优化器的beta1超参数
```

实验步骤-数据下载与处理

- 步骤四： 定义`create\_dataset\_imagenet`函数对数据进行处理和增强操作。（代码参照实验手册第7页2.2章步骤4）

```
def create_dataset_imagenet(dataset_path):  
    """数据加载"""  
    dataset = ds.ImageFolderDataset(dataset_path,  
                                     num_parallel_workers=4,  
                                     shuffle=True,  
                                     decode=True)
```

```
# 数据增强操作  
transforms = [  
    vision.Resize(image_size),  
    vision.CenterCrop(image_size),  
    vision.HWC2CHW(),  
    lambda x: ((x / 255).astype("float32"))  
].....
```

实验步骤-数据下载与处理

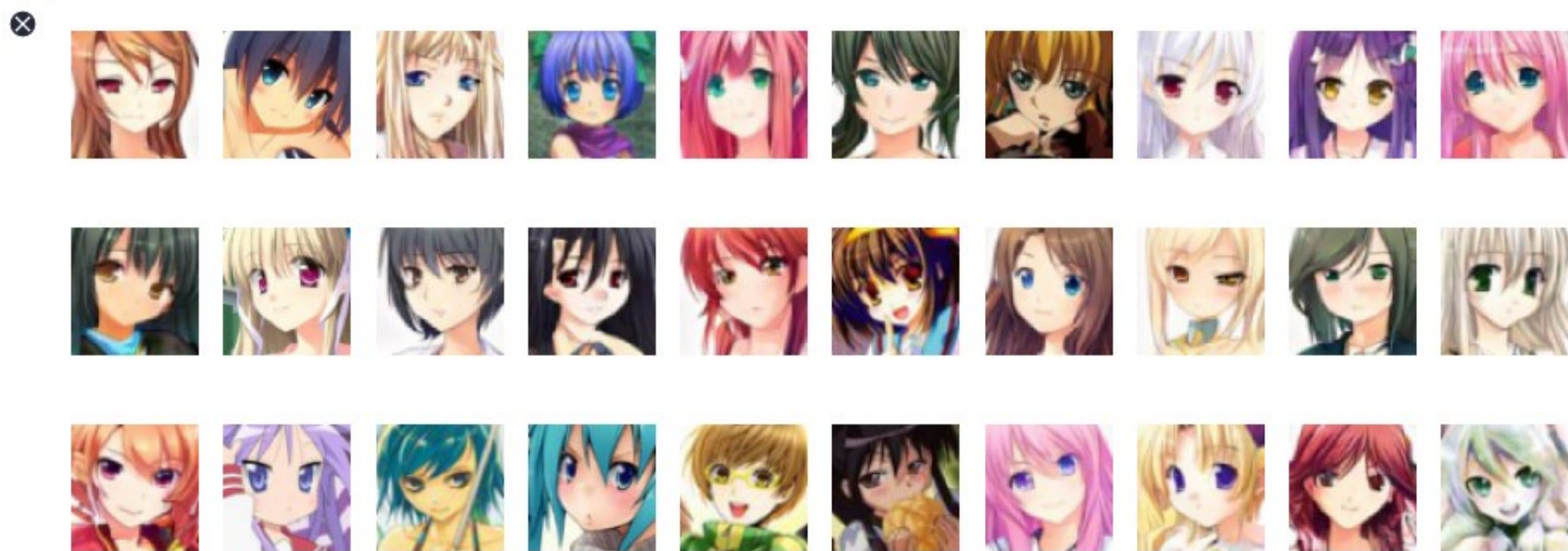
- 步骤五：通过create_dict_iterator函数将数据转换成字典迭代器，然后使用`matplotlib`模块可视化部分训练数据。（代码参照实验手册第8页2.2章步骤5）

```
def plot_data(data):  
    # 可视化部分训练数据  
  
    plt.figure(figsize=(10, 3), dpi=140)  
  
    for i, image in enumerate(data[0][:30], 1):  
        plt.subplot(3, 10, i)  
        plt.axis("off")  
        plt.imshow(image.transpose(1, 2, 0))  
  
    plt.show()
```

```
sample_data =  
next(dataset.create_tuple_iterator(output_numpy=  
True))  
  
plot_data(sample_data)
```

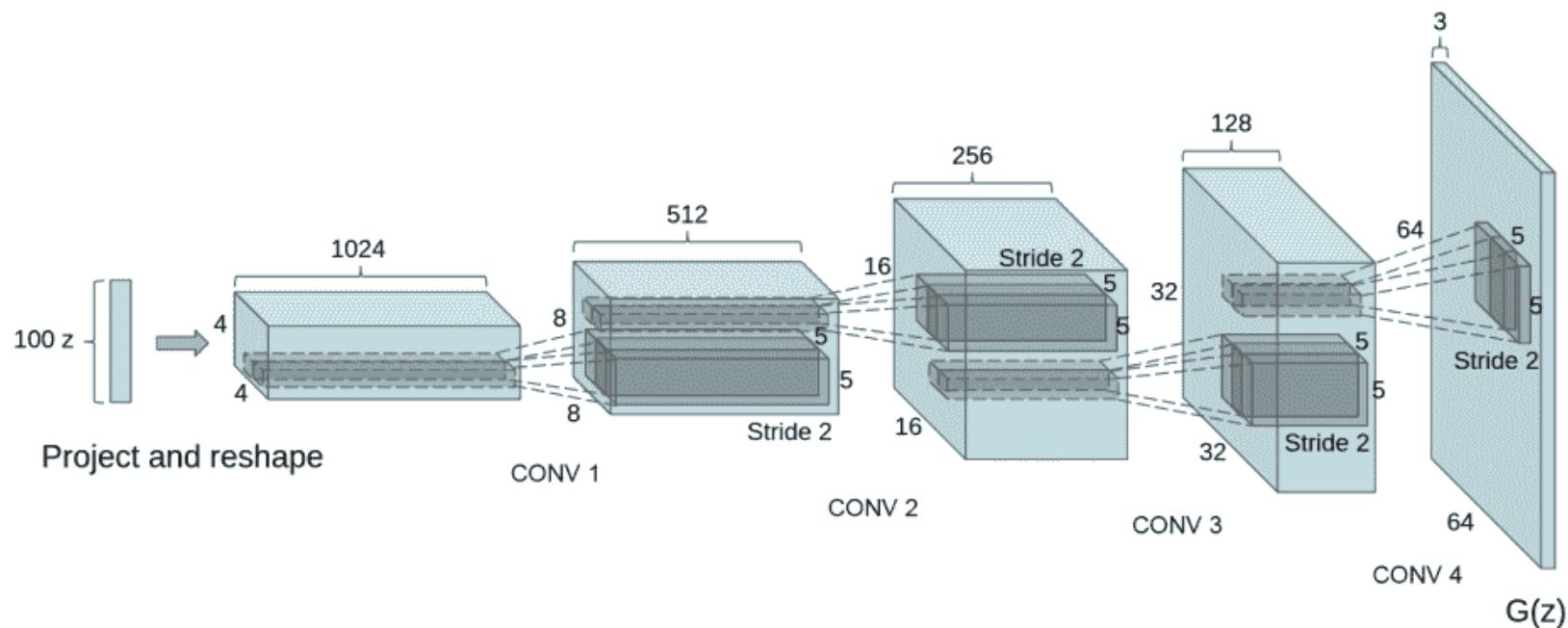

实验步骤-数据下载与处理

- 步骤五：通过create_dict_iterator函数将数据转换成字典迭代器，然后使用`matplotlib`模块可视化部分训练数据。
- 可视化的训练数据图：



实验步骤-创建网络

- 步骤一：生成器
- 生成器G的功能是将隐向量 z 映射到数据空间。由于数据是图像，这一过程也会创建与真实图像大小相同的 RGB 图像。在实践场景中，该功能是通过一系列 Conv2dTranspose 转置卷积层来完成的，每个层都与 BatchNorm2d 层和 ReLU 激活层配对，输出数据会经过 tanh 函数，使其返回 $[-1,1]$ 的数据范围内。



实验步骤-创建网络

- 步骤一：生成器的代码实现：（代码参照实验手册第9页2.3章步骤1）

```
weight_init = Normal(mean=0, sigma=0.02)
gamma_init = Normal(mean=1, sigma=0.02)

class Generator(nn.Cell):
    """DCGAN网络生成器"""
    def __init__(self):
        super(Generator, self).__init__()
        self.generator = nn.SequentialCell(
            .....
generator = Generator()
```

实验步骤-创建网络

- 步骤二：判别器（代码参照实验手册第10页2.3章步骤2）
- 如前所述，判别器D是一个二分类网络模型，输出判定该图像为真实图的概率。通过一系列的Conv2d、BatchNorm2d和LeakyReLU层对其进行处理，最后通过Sigmoid激活函数得到最终概率。
- DCGAN论文提到，使用卷积而不是通过池化来进行下采样是一个好方法，因为它可以让网络学习自己的池化特征。
- 判别器的代码实现如下：

```
class Discriminator(nn.Cell):  
    """DCGAN网络判别器"""  
    def __init__(self):  
        .....  
discriminator = Discriminator()
```

实验步骤-损失函数和优化器

- 当定义了D和G后，接下来将使用MindSpore中定义的二进制交叉熵损失函数 BCELoss，为D和G加上损失函数和优化器。（代码参照实验手册第11页2.4章）

```
# 定义损失函数
```

```
adversarial_loss = nn.BCELoss(reduction='mean')
```

- 这里设置了两个单独的优化器，一个用于D，另一个用于G。这两个都是 $lr = 0.0002$ 和 $\beta_1 = 0.5$ 的Adam优化器。

```
# 为生成器和判别器设置优化器
```

```
optimizer_D = nn.Adam(discriminator.trainable_params(), learning_rate=lr, beta1=beta1)
```

```
optimizer_G = nn.Adam(generator.trainable_params(), learning_rate=lr, beta1=beta1)
```

```
optimizer_G.update_parameters_name('optim_g.')
```

```
optimizer_D.update_parameters_name('optim_d.')
```

实验步骤-模型训练

- 训练分为两个主要部分：训练判别器和训练生成器。
- 训练判别器：
 - 训练判别器的目的是最大程度地提高判别图像真伪的概率。按照Goodfellow的方法，是希望通过提高其随机梯度来更新判别器，所以我们要最大化 $\log D(x) + \log(1 - D(G(z)))$ 的值。
- 训练生成器
 - 如DCGAN论文所述，我们希望通过最小化 $\log(1 - D(G(z)))$ 来训练生成器，以产生更好的虚假图像。
 - 在这两个部分中，分别获取训练过程中的损失，并在每个周期结束时进行统计，将fixed_noise批量推送到生成器中，以直观地跟踪G的训练进度。

实验步骤-模型训练

- 步骤一：下面实现模型训练正向逻辑：（代码参照实验手册第12页2.5章步骤1）

```
.....

grad_generator_fn = ms.value_and_grad(generator_forward, None,
                                       optimizer_G.parameters,
                                       has_aux=True)

grad_discriminator_fn = ms.value_and_grad(discriminator_forward, None,
                                          optimizer_D.parameters)

@ms.jit
def train_step(imgs):
    .....
```

实验步骤-模型训练

- 步骤二：循环训练网络，每经过50次迭代，就收集生成器和判别器的损失，以便于后面绘制训练过程中损失函数的图像。（代码参照实验手册第13页2.5章步骤2）

```
total = dataset.get_dataset_size()
for epoch in range(num_epochs):
    generator.set_train()
    discriminator.set_train()
    .....
```

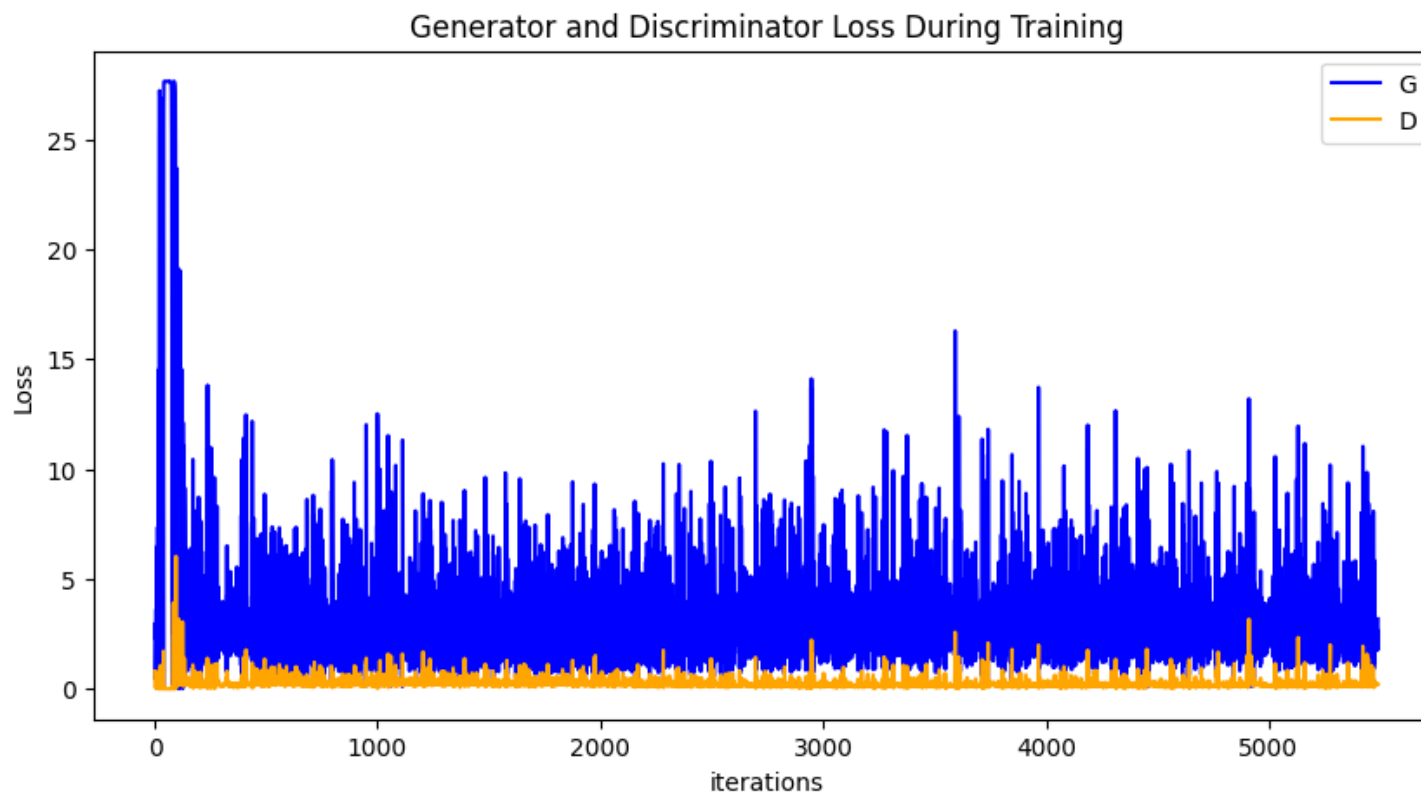

实验步骤-模型训练

- 步骤二：完整的10个epoch模型训练结果，如下：

```
[ 1/10][ 1/549] Loss_D: 0.8013 Loss_G: 0.5065
[ 1/10][101/549] Loss_D: 0.1116 Loss_G:13.0030
[ 1/10][201/549] Loss_D: 0.1037 Loss_G: 2.5631
...
[ 1/10][401/549] Loss_D: 0.6240 Loss_G: 0.5548
[ 1/10][501/549] Loss_D: 0.3345 Loss_G: 1.6001
[ 1/10][549/549] Loss_D: 0.4250 Loss_G: 1.1978
...
[10/10][501/549] Loss_D: 0.2898 Loss_G: 1.5352
[10/10][549/549] Loss_D: 0.2120 Loss_G: 3.1816
```

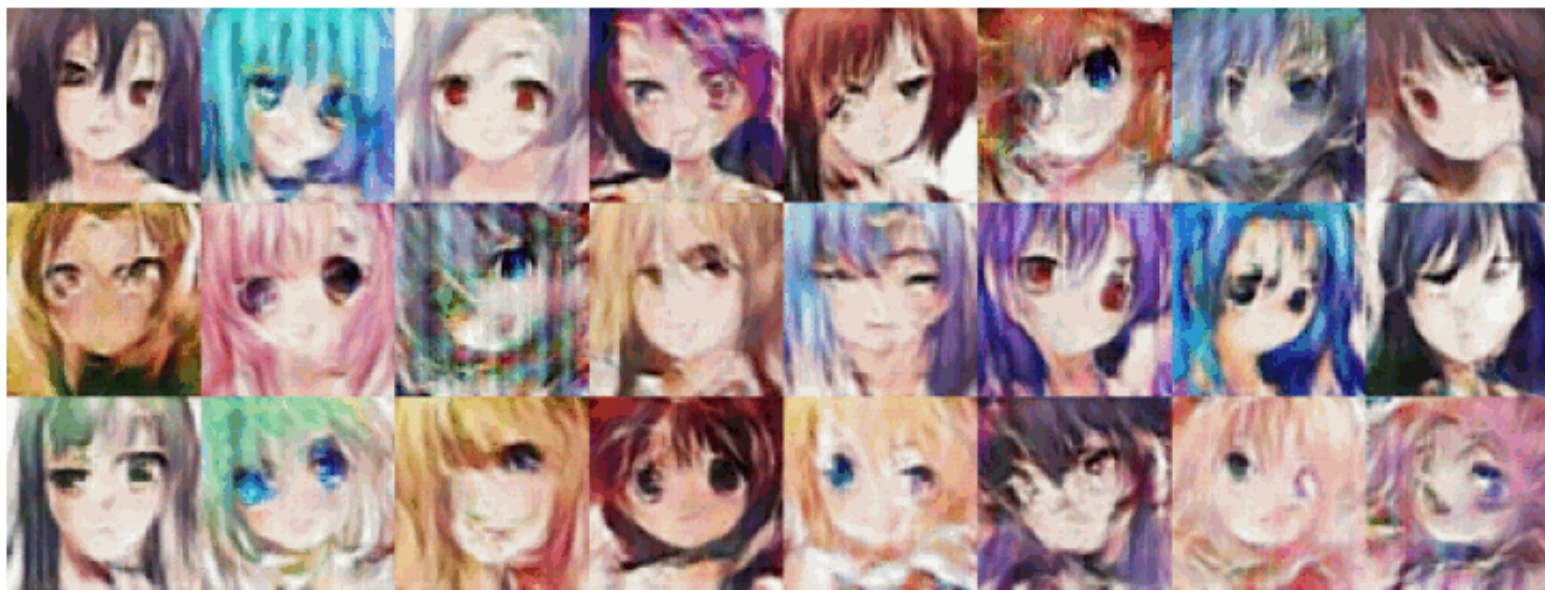
实验步骤-结果展示

- 步骤一：描绘‘D’和‘G’损失与训练迭代的关系图：（代码参照实验手册第14页2.6章步骤1）



实验步骤-结果展示

- 步骤二：可视化训练过程中通过隐向量`fixed\_noise`生成的图像。
(代码参照实验手册第15页2.6章步骤2)



- 从上面的图像可以看出，随着训练次数的增多，图像质量也越来越好。如果增大训练周期数，当num_epochs达到50以上时，生成的动漫头像图片与数据集中的较为相似。